



Más allá del Transformer: nuevas arquitecturas para el razonamiento en IA

Antonio Miguel Artiaga

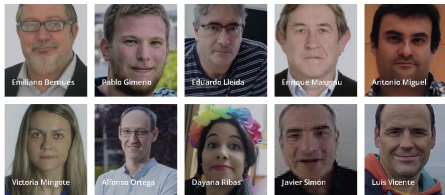
Introducción

VIVOLAB

HOME RESEARCH PROJECTS TEAM PUBLICATIONS DEMOS NEWS INTRANET WIKI

TEAM

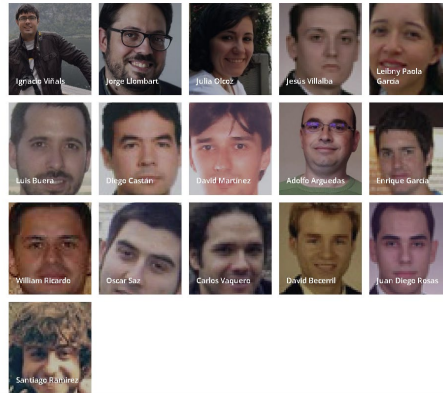
Researchers



Students



Former Members



- Más de 20 años desarrollando tecnologías y sistemas de Inteligencia Artificial aplicados a la **señal de voz y el procesamiento del lenguaje**
- Experiencia en la formación de profesionales especializados en Aprendizaje Automático, Procesamiento de Señales e Inteligencia Artificial
- Transferencia de conocimiento desde la academia al sector industrial de I+D

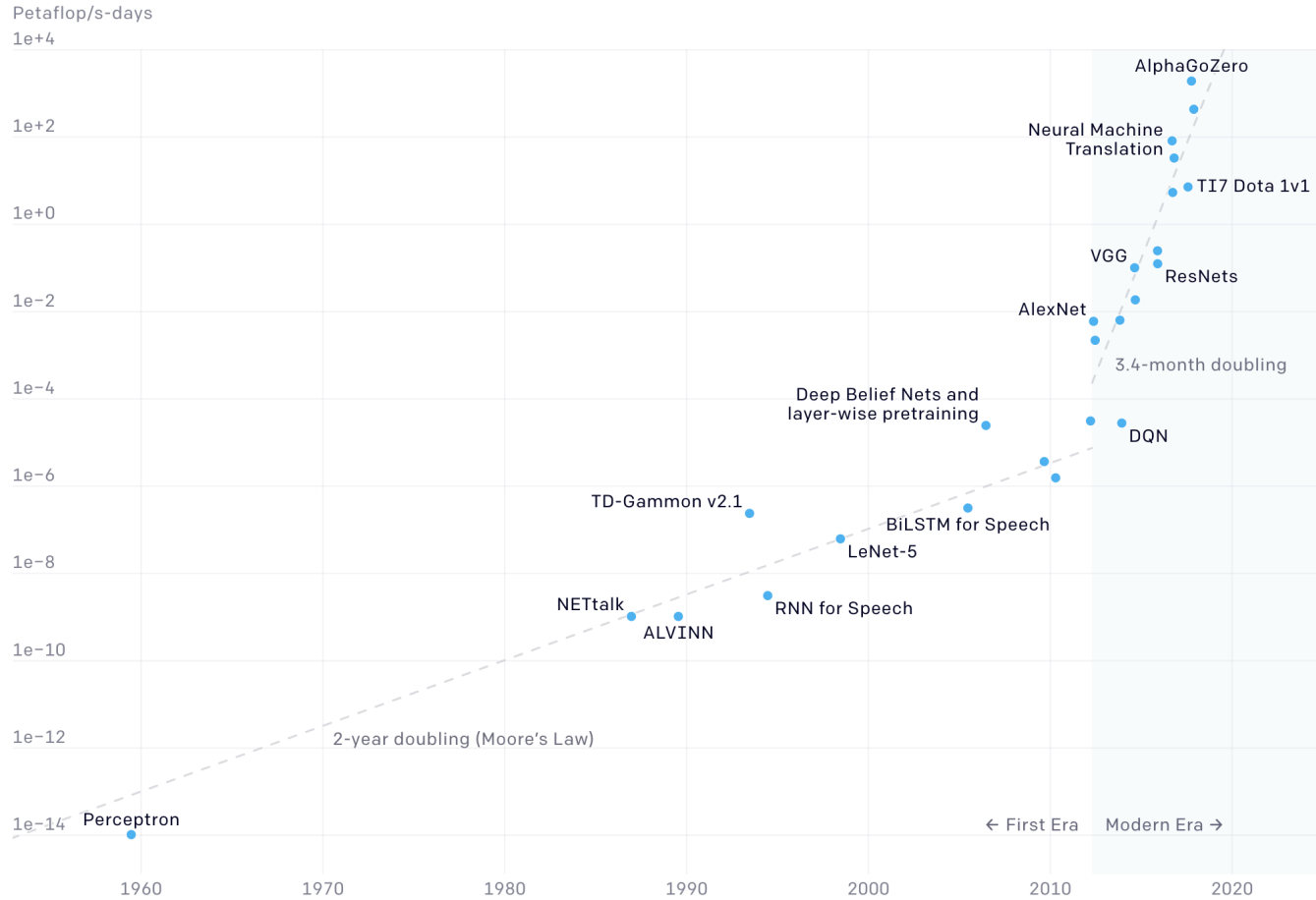
<https://vivolab.i3a.es/>

Antecedentes

Antecedentes

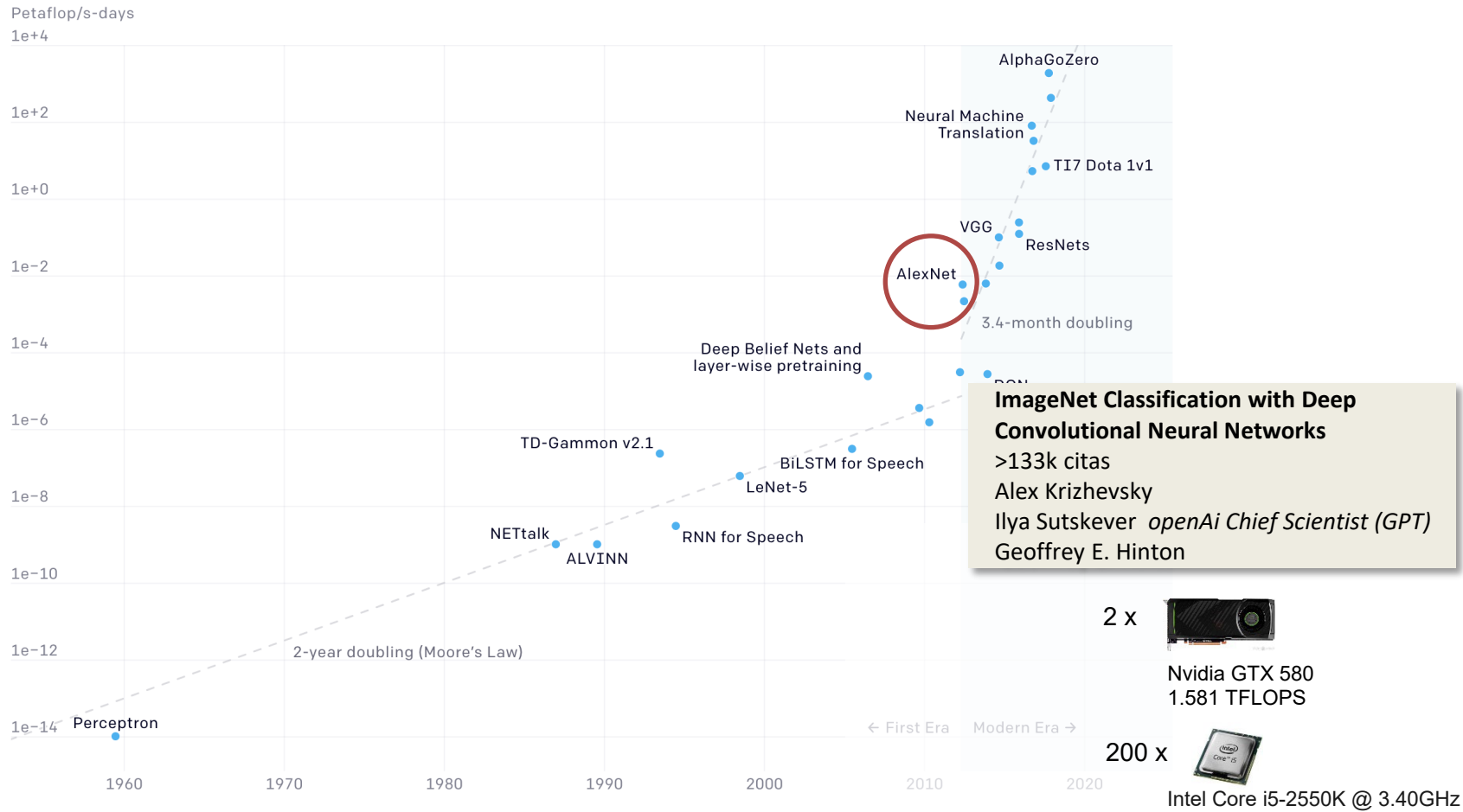
Two Distinct Eras of Compute Usage in Training AI Systems

<https://openai.com/research/ai-and-compute>



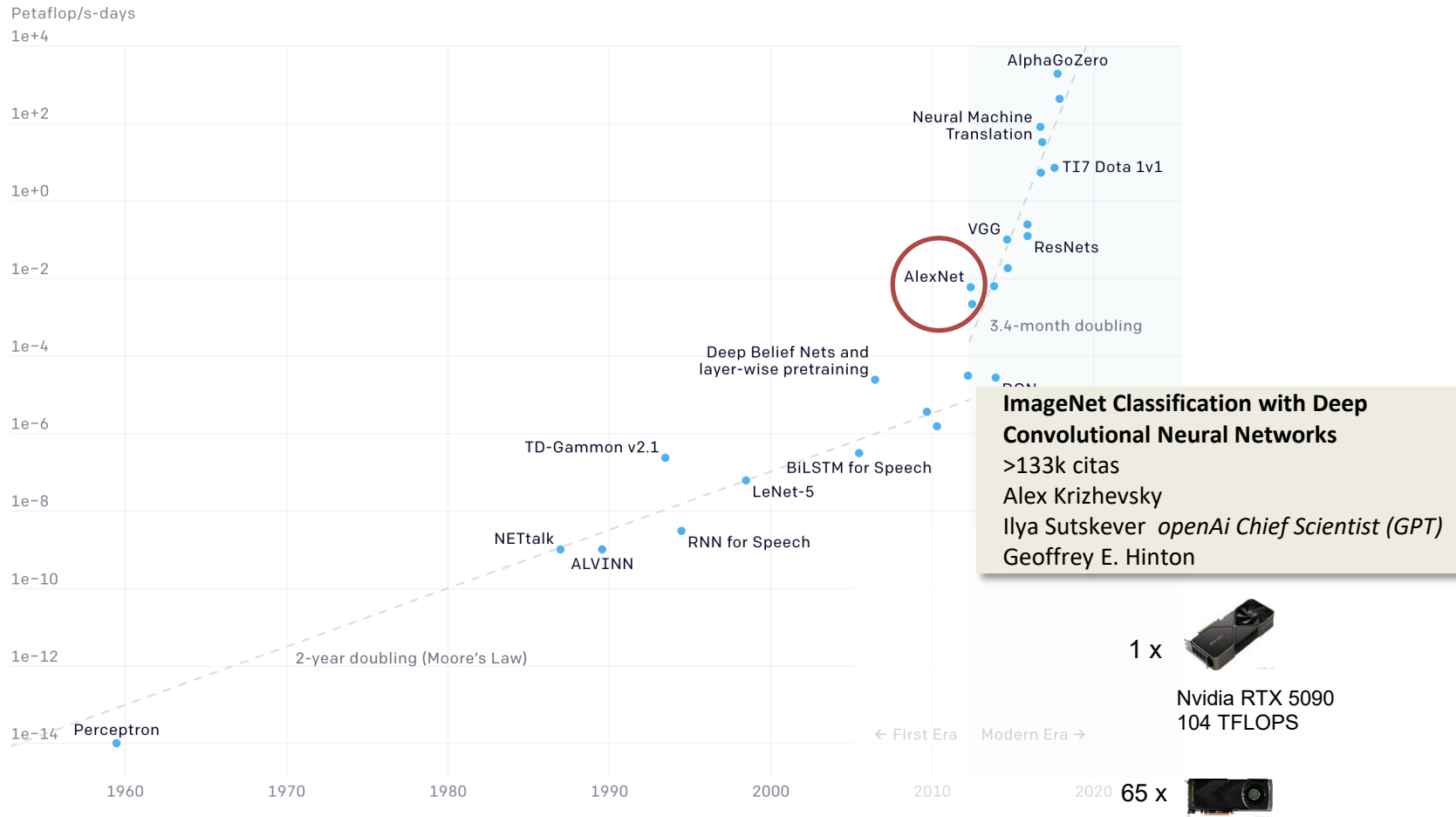
Antecedentes

Two Distinct Eras of Compute Usage in Training AI Systems



Antecedentes

Two Distinct Eras of Compute Usage in Training AI Systems



Antecedentes

Nvidia RTX 5090, 2025
 104 TFlops (~4160 x intel i5)



Playstation 4s, 2016
 1.8 TFlops (~90 x intel i5)



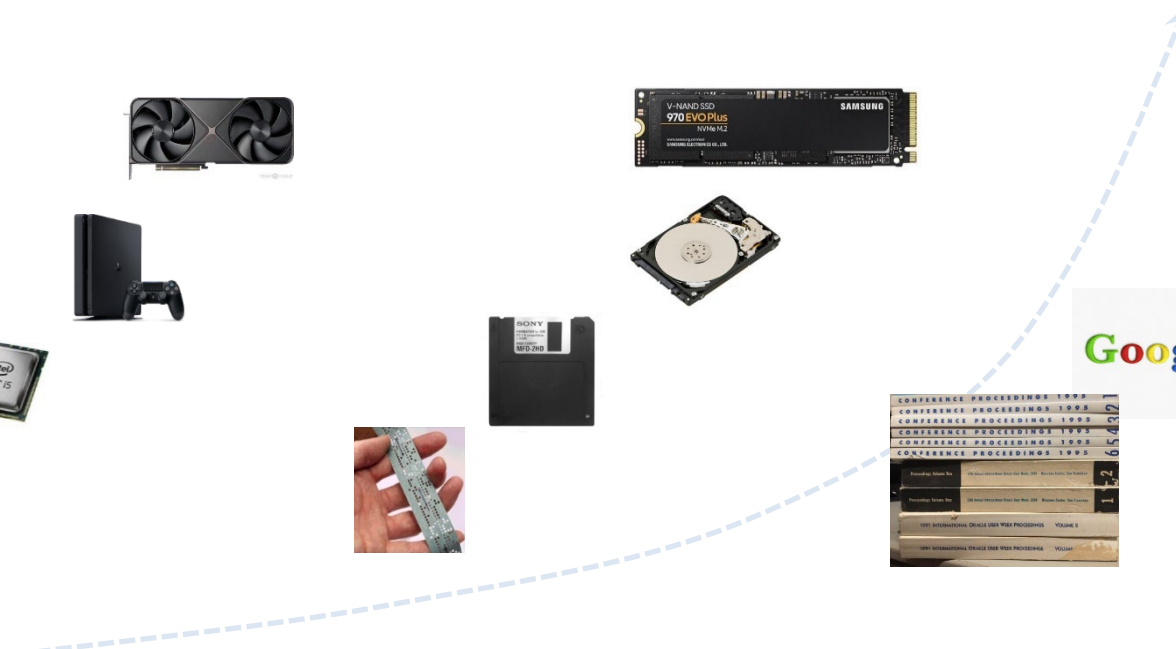
Intel i5, 2018
 25 mil millones de FLOPS



Intel 8086, 1978
 50 mil operaciones por segundo



PYTORCH



Antecedentes

Company/Entity	H100 GPU Count	Type
Meta	350,000	Private cloud
XAI/X	100,000	Private cloud
Tesla	35,000	Private cloud
Lambda	30,000	Public cloud
Google A3	26,000	Public cloud
Oracle Cloud	16,000	Public cloud
Poolside	10,000	Public cloud
Magic	8,000	Public cloud
Andromeda	3,632	Private cloud
Scaleway	1,016	Private cloud
Hugging Face	768	Public cloud
DeepL	544	Private cloud
Recursion	504	Private cloud
Princeton	300	National HPC
Photroom	256	Private cloud

Año 3 AG (after GPT)

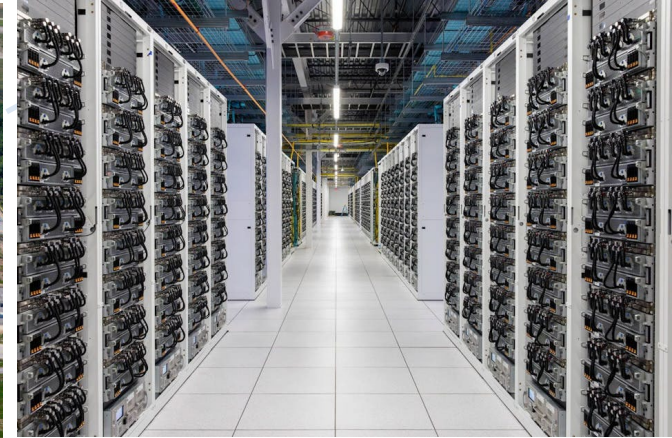
T4
A100
H100
H200

FP16 65.13 TFLOPS (~2600 x intel i5)

FP16 77.97 TFLOPS (~3120 x intel i5)

FP16 204.9 TFLOPS (~8196 x intel i5)

FP16 241.3 TFLOPS (~9640 x intel i5)

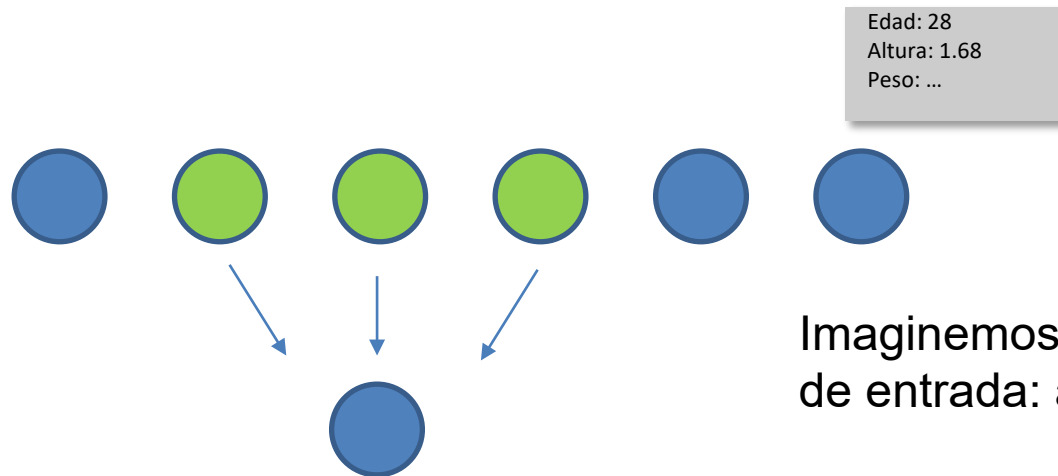


xAI Colossus Supercomputer Data Center – Memphis USA

Fundamentos

Fundamentos

- Procesado en fases, capas

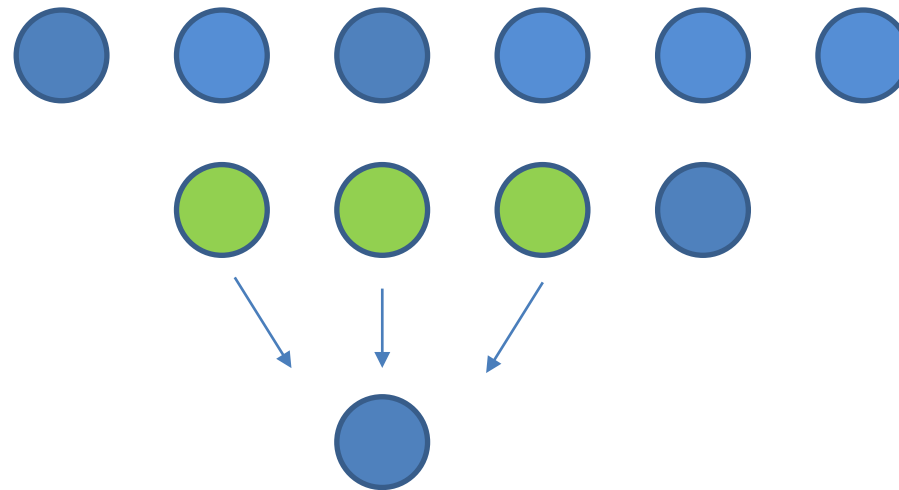


Imaginemos unos datos de entrada: altura, edad,..

*Por ejemplo podríamos decir:
 Pregunta a los tres que tengas
 en la fila de delante y quédate
 con el máximo, mínimo, etc*

Fundamentos

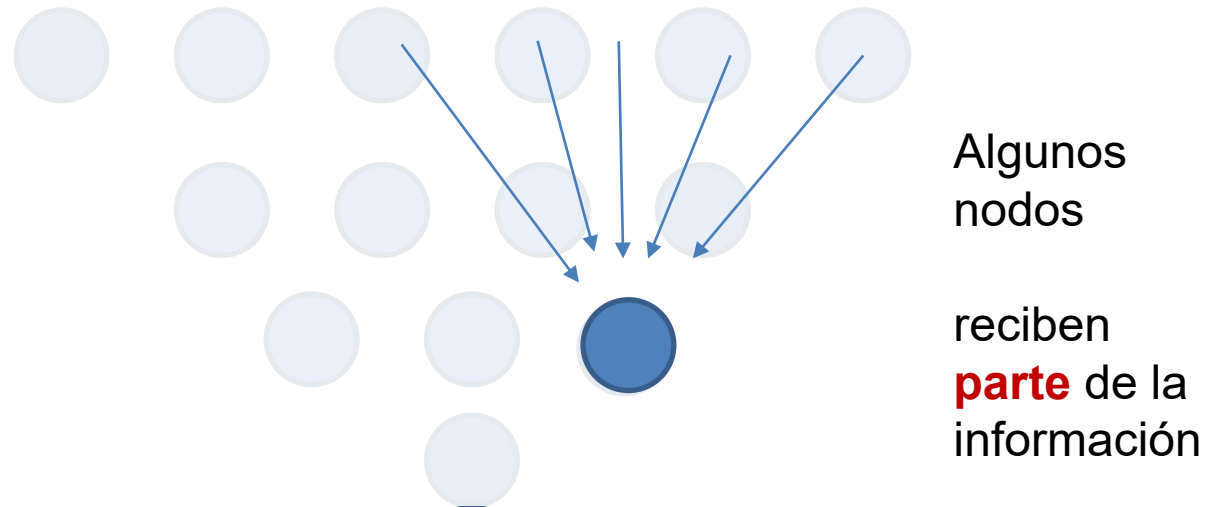
- Procesado en fases, capas



Y repetimos,
en todas las
filas...

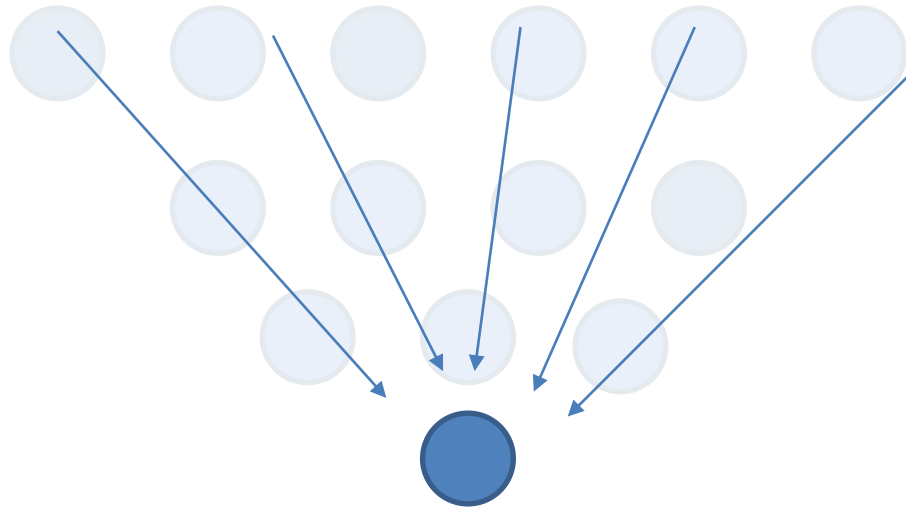
Fundamentos

- Procesado en fases, capas



Fundamentos

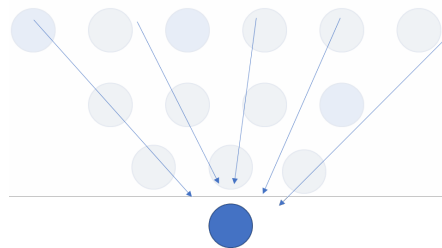
- Procesado en fases, capas



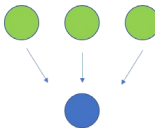
Ahora al último
le podríamos
preguntar,
**¿quién es el
más joven ?**
Ha recibido
toda la
información

Fundamentos

- Procesado en fases, capas

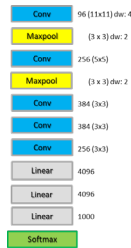


- En una red convolucional cada píxel es procesado por una entrada
- La información se va combinando en capas sucesivas (capas/layers)
- Cada unidad está conecta a varias unidades de las etapas anteriores



Fundamentos

- Deep Learning: Ejemplo redes convolucionales
 - A partir de 2012 se generó un gran interés por aumentar la profundidad de las redes (número de capas)



2012: **Alexnet** 7 capas
 84.6 % aciertos (Top 5)
 clasificación de imagen (Imagenet)



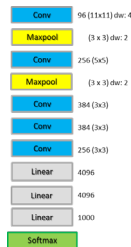
2014: **Inception** 25 capas
 93.3% aciertos



2015: **Resnet** >100 capas
 96.43% aciertos

Fundamentos

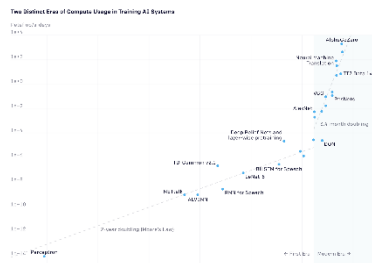
- Deep Learning: Ejemplo redes convolucionales
 - A partir de 2012 se generó un gran interés por aumentar la profundidad de las redes (número de capas)



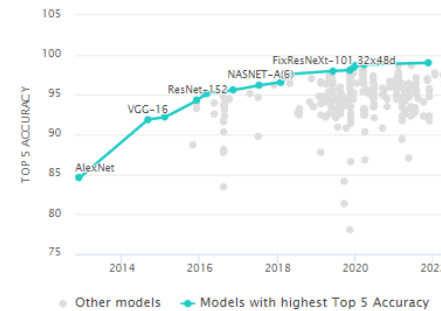
2012: **Alexnet** 7 capas
 84.6 % aciertos (Top 5)
 clasificación de imagen (Imagenet)

2014: **Inception** 25 capas
 93.3% aciertos

2015: **Resnet** >100 capas
 96.43% aciertos



recursos

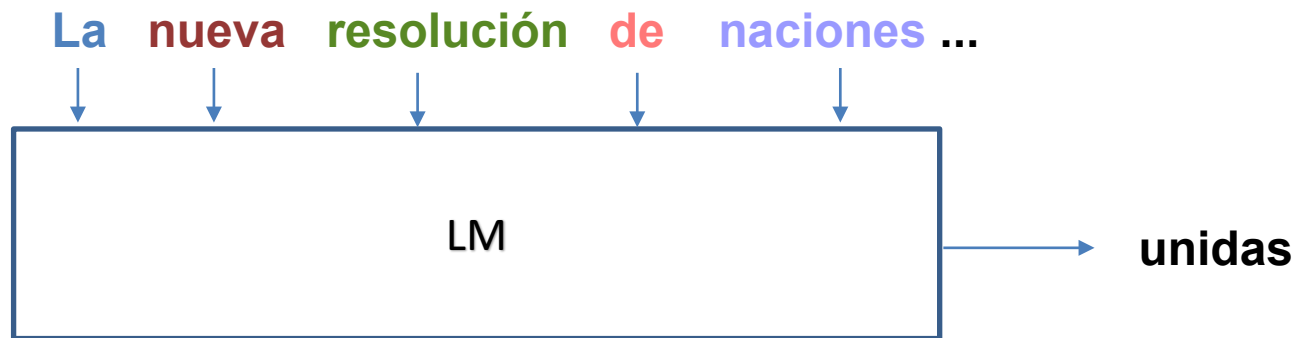


rendimiento

Fundamentos

- Modelo de lenguaje: objetivo

predecir la siguiente palabra en una secuencia de palabras (un texto)



Fundamentos

• Representación de palabras (ejemplo simplificado)

- Cada palabra del vocabulario representada por un **vector** (32k por ejemplo en LLAMA)

palabra	rey	reina	gato	árbol	ráiz	gustará	odiaba

propiedades							
persona	☒	☒					
monarquía	☒	☒					
vegetal				☒	☒		
animal			☒	☒			
masculino	☒		☒	☒			
femenino		☒			☒		
positivo						☒	
negativo							☒
verbo						☒	☒
presente							
pasado							☒
futuro					☒		

Un cambio de significado puede ir asociado a cambiar uno de esos aspectos, **recuerdan más a un ideograma chino** que a una representación mediante un alfabeto fonético

木 → 本
Árbol → Raíz

- palabras parecidas **representaciones similares**
- **transformar** una palabra para llegar a otra

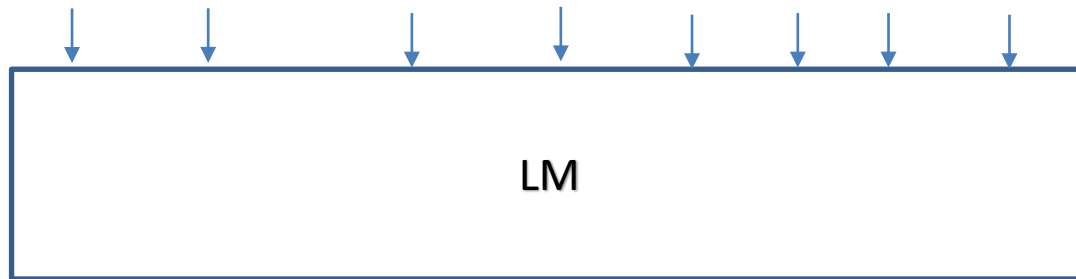
rey - hombre + mujer	-> reina
tenis - raqueta + bate	-> béisbol
motocicleta - motor + pedales	-> bicicleta
amar - positivo + negativo	-> odiar
parís - francia + españa	-> madrid

Fundamentos

- Modelo de lenguaje: partes de palabra (tokens)

Modelo LLAMA 7B
32k tokens

_La _nueva _resol ución _de _n acciones...



$p(\text{unidas} \mid \text{_La, _nueva, _resol, ución, _de, _n, acciones})$

- Lo que obtendremos es una **estimación de la probabilidad**

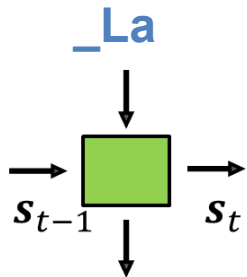
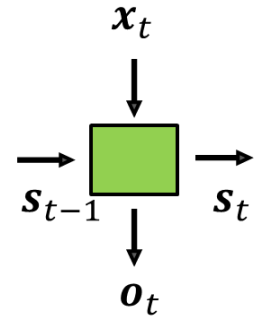
token		prob

_un		0.48
_y		0.04
_en		0.03
_Un		0.03
_que		0.02
,		0.02
...		

Fundamentos

• Redes recurrentes

- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780
 - **Limitación:** cada celda tiene una dimensión finita para recibir información de los instantes previos y escribir nueva información para el futuro

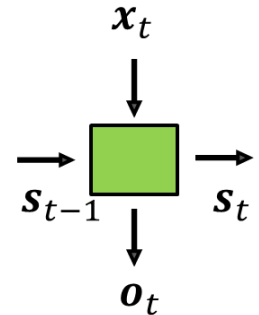


token	prob
unch	(0.21)
uren	(0.09)
wn	(0.03)
ure	(0.03)
...	

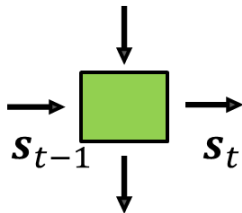
Fundamentos

• Redes recurrentes

- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780
 - Limitación:** cada celda tiene una dimensión finita para recibir información de los instantes previos y escribir nueva información para el futuro



_La _nueva



token	prob

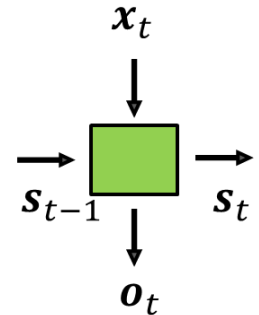
_versión	(0.04)
_gener	(0.03)
_est	(0.02)
_c	(0.01)
...	

Fundamentos

• Redes recurrentes

• Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8), 1735-1780

- **Limitación:** cada celda tiene una dimensión finita para recibir información de los instantes previos y escribir nueva información para el futuro



2015 <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

- Red recurrente (LSTM) 3 capas dimensión 512, **5.8M**
- **4.4MB** texto Shakespeare

DUKE VINCENTIO: Well, your wit is in the care of side and that.

Second Lord: They would be ruled after this chamber, and my fair nues begun out of the fact, to be conveyed, Whose noble souls I'll have the heart of the wars.

Clown: Come, sir, I will make did behold your worship.

VIOLA: I'll drink it.

2016 <https://medium.com/deep-writing/harry-potter-written-by-artificial-intelligence-8a9431803da6>

- Primeros 4 libros

Ron didn't even upset her little ingredients on the toilet, and a group of third-year girls last year. Highly bushy and then burst away from them quickly.

"Thought you're all right?" he said.

Harry grinned at Harry. "Why should she be cheerful so while you gave detentions, Moody!"

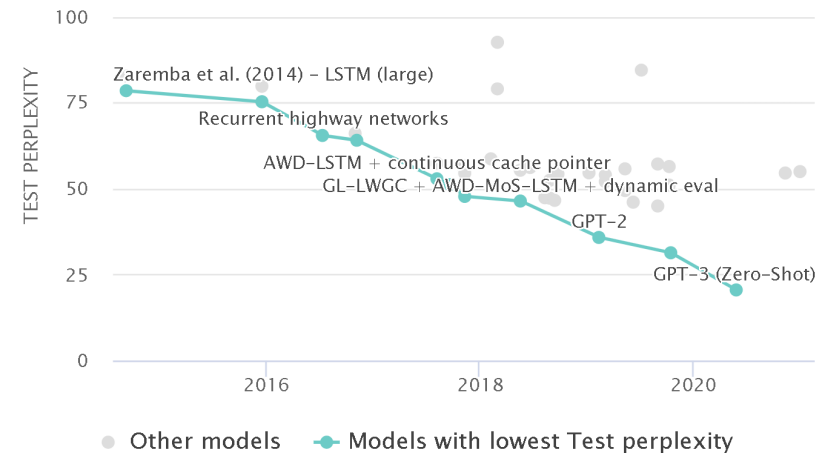
Transformer

Fundamentos

- Transformers
 - Modelo presentado por Google en 2017
 - “Attention is all you need” >200k citas

Attention Is All You Need

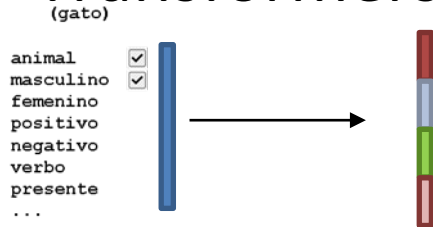
Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com	Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu	Łukasz Kaiser* Google Brain lukaszkaier@google.com	
Illia Polosukhin* † illia.polosukhin@gmail.com			



- ¿Por qué ha tenido tanto éxito?

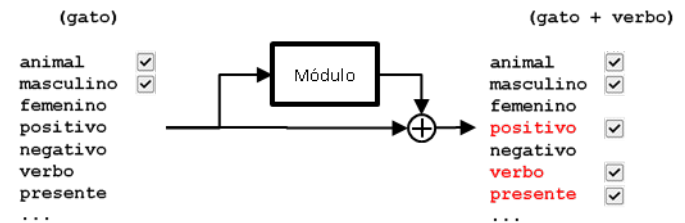
Fundamentos

Transformers: 4 ideas



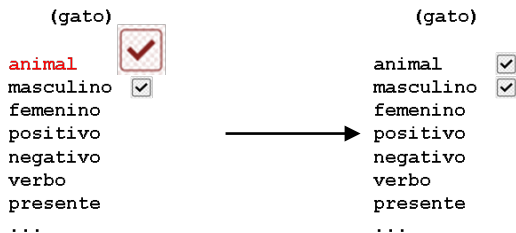
Los vectores de representación se separan en vectores más pequeños (registros/variables) “heads”

- **Operaciones independientes** en cada uno
- Permite la manipulación más precisa y en **paralelo**

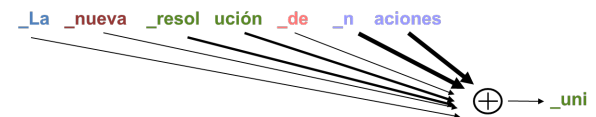


La información se procesa **incrementalmente** (residual)

Thought vectors (G. Hinton)



Es crítico para que funcione **normalizar la escala** de los vectores



Dos tipos de módulos

Procesado de vectores individuales

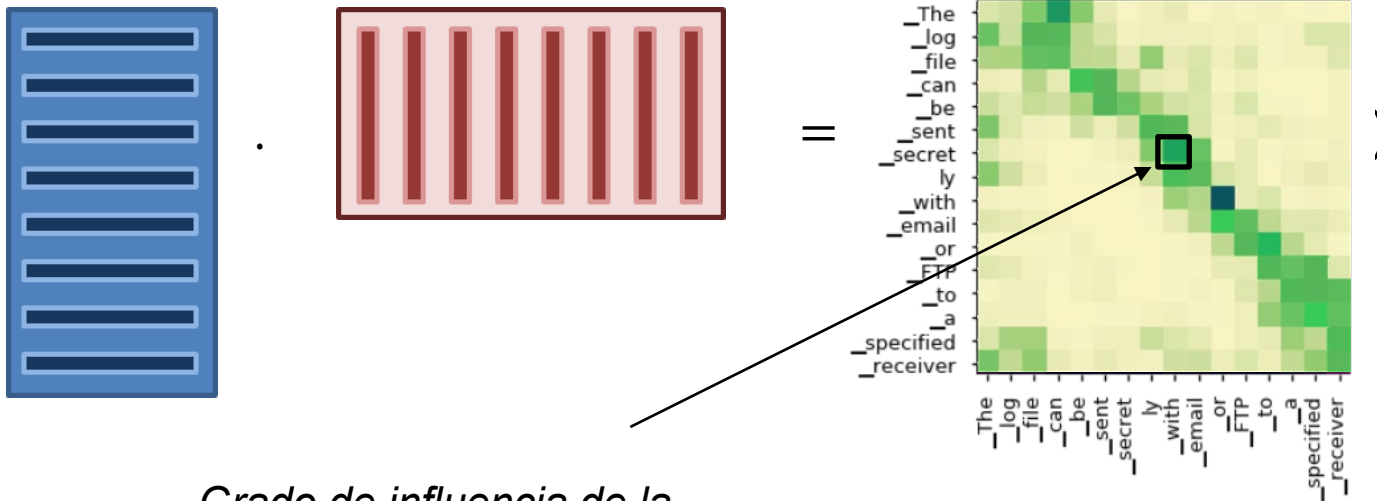
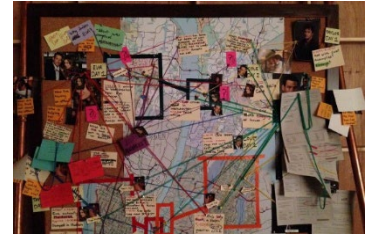
- red neuronal

Mezcla de vectores

- **mecanismo de atención** (auto)
(suma ponderada)

Fundamentos

- Mecanismo de atención (auto)
 - Son capaces de analizar la relación de todas las palabras
 - ¿Cómo se genera esas relaciones?



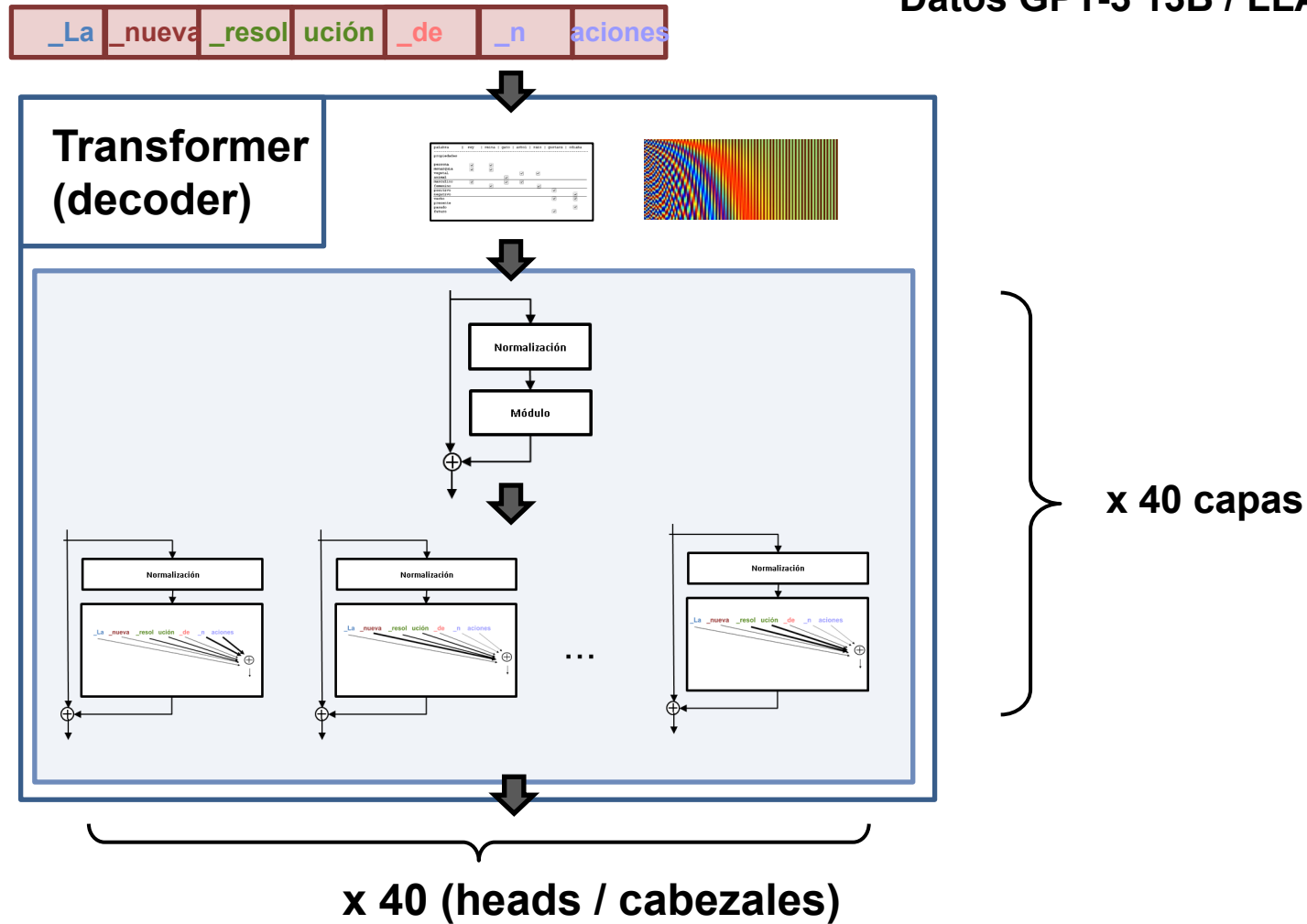
Grado de influencia de la
entrada i sobre la j

<https://nlp.seas.harvard.edu/2018/04/03/attention.html>

RNN

Fundamentos

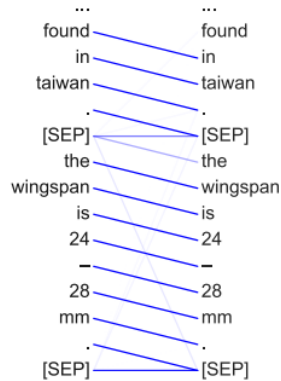
Datos GPT-3 13B / LLAMA13B



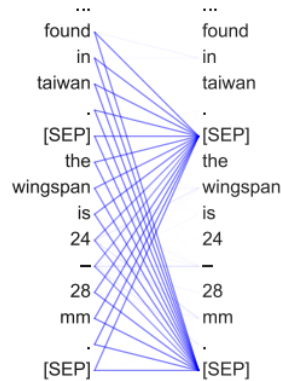
Fundamentos

• Transformers

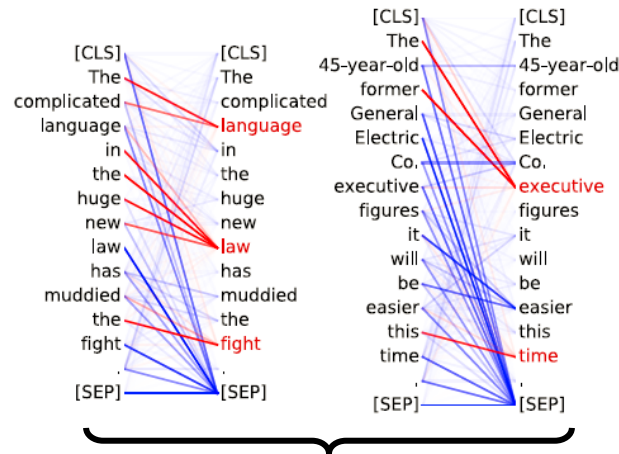
- *Son capaces de analizar la relación de todas las entradas, múltiples aspectos: heads (cabezales)*
 - *por ejemplo en LLAMA 7b -> 32 (en 32 capas)*



palabra anterior



Final de frase



Determinantes y modificadores de un nombre



Desambigüación ellos -> vecinos

<https://medium.com/dair-ai/aspects-of-language-captured-by-bert-32bc3c54016f>

<https://medium.com/synapse-dev/understanding-bert-transformer-attention-isnt-all-you-need-5839ebd396db>

RNN

VIVOLAB

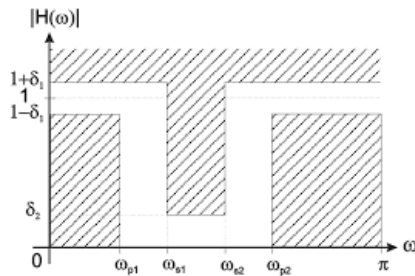


Universidad Zaragoza

Algoritmos y computación

Transformers

- Transformers: ¿Máquinas de propósito general?
 - Planteamiento clásico de un problema de ingeniería

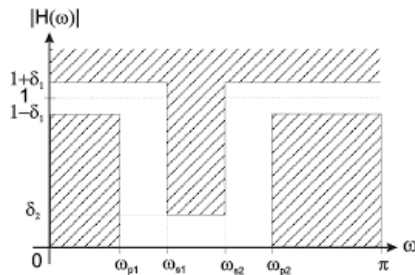


- se plantean unos **requisitos**/restricciones
- se elige un **método** para la solución, a veces de forma analítica, otras aproximada, otras por optimización iterativa...
- se **comprueba** si la solución cumple con las restricciones

Transformers

- Transformers: ¿Máquinas de propósito general?

- Planteamiento clásico de un problema de ingeniería



- se plantean unos requisitos/restricciones
- se elige un método para la solución, a veces de forma analítica, otras aproximada, otras por optimización iterativa...
- se comprueba si la solución cumple con las restricciones

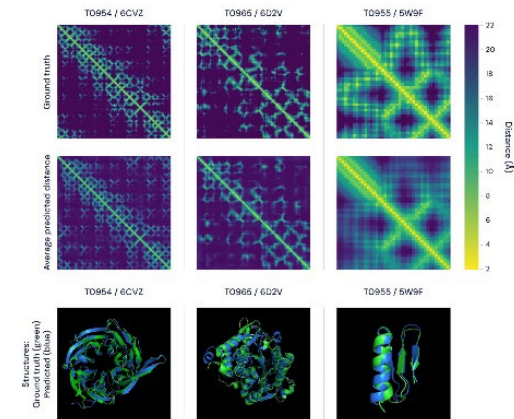
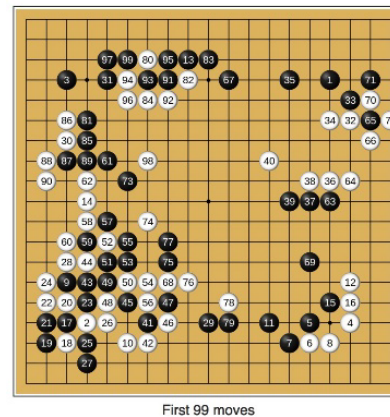
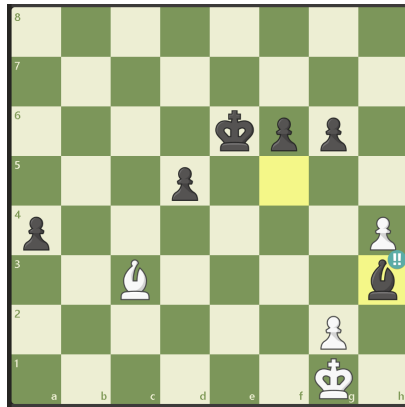
- En aprendizaje automático presentamos datos y la solución esperada

entrada	salida
0, 2, 1, 5	0, 1, 2, 5
4, 5, 3	3, 4, 5
0, 1	0, 1
8, 4, 5	4, 5, 8
...	

- Por ejemplo dar varios números aleatorios y como objetivo querer los números ordenados
- **Preparamos muchos ejemplos** distintos y entrenamos un sistema
- Medimos la calidad, **no siempre acierta el 100%** de los casos!

Transformers

- ¿Qué utilidad tiene aprenda un algoritmo que ya existe? (y que no siempre lo haga bien)
 - Estos experimentos se han hecho para demostrar esa capacidad de generar pequeños algoritmos si se necesitan
 - Estas capacidad destaca cuando **no se conoce un algoritmo** para resolver la tarea: traducir entre idiomas, resumir textos, evaluar una posición estratégica, predecir el plegado de proteínas...



<https://www.deepmind.com/blog/alphafold-using-ai-for-scientific-discovery-2020>

Transformers

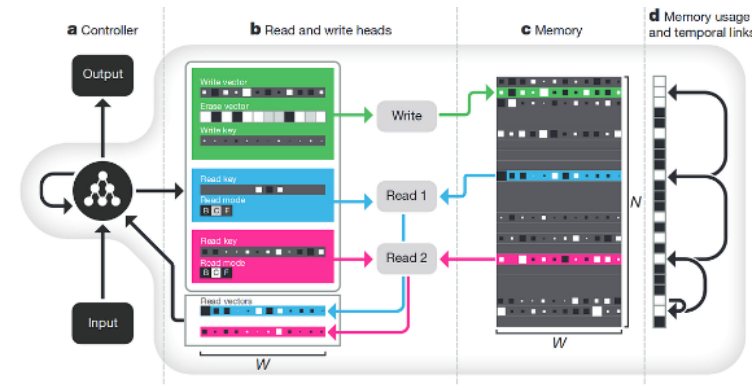
- Transformers: ¿Máquinas programables de propósito general?
 - “differentiable computer”
 - 2014, Neural Turing Machines
 - 2016, Differentiable Neural Computer

nature

Hybrid computing using a neural network with dynamic external memory

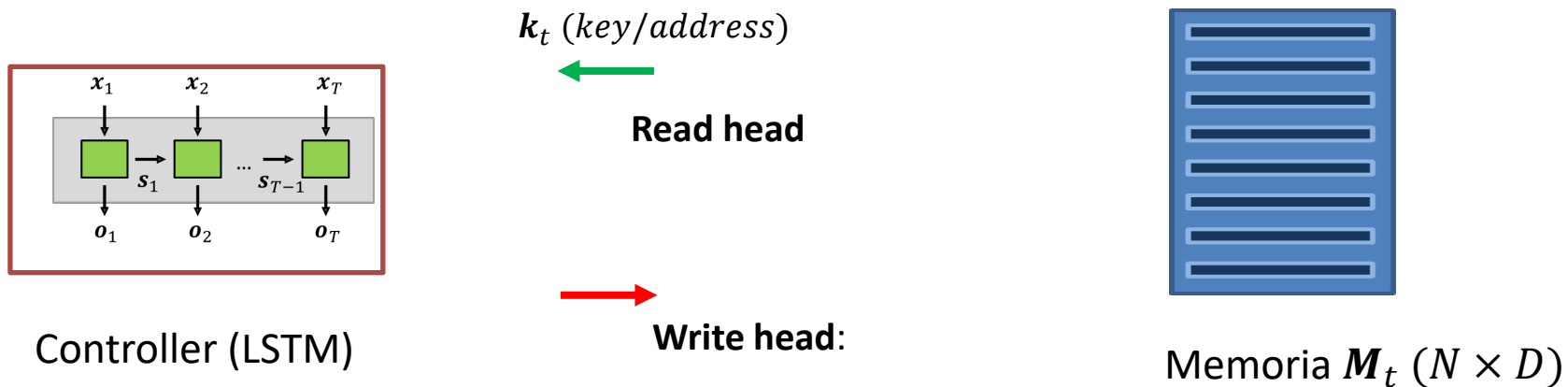
[Alex Graves](#), [Greg Wayne](#), [Malcolm Reynolds](#), [Tim Harley](#), [Ivo Danihelka](#), [Agnieszka Grabska-Barwińska](#), [Sergio Gómez Colmenarejo](#), [Edward Grefenstette](#), [Tiago Ramalho](#), [John Agapiou](#), [Adrià Puigdomènech Badia](#), [Karl Moritz Hermann](#), [Yori Zwols](#), [Georg Ostrovski](#), [Adam Cain](#), [Helen King](#), [Christopher Summerfield](#), [Phil Blunsom](#), [Koray Kavukcuoglu](#) & [Demis Hassabis](#)

- Tenía memorías diferenciables:
 - » Escritura / Lectura
 - » **Heads / cabezales**
- 2017, Transformers



Transformers

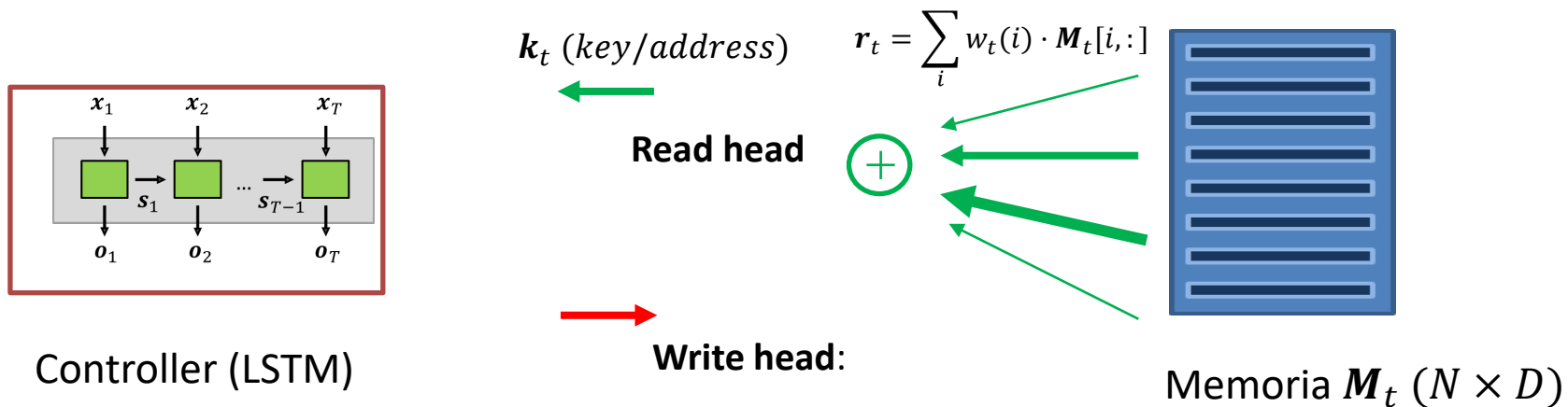
- Transformers: ¿Máquinas programables de propósito general?
 - “differentiable computer”



w_t peso/dirección de lectura/escritura se calcula por **parecido**

Transformers

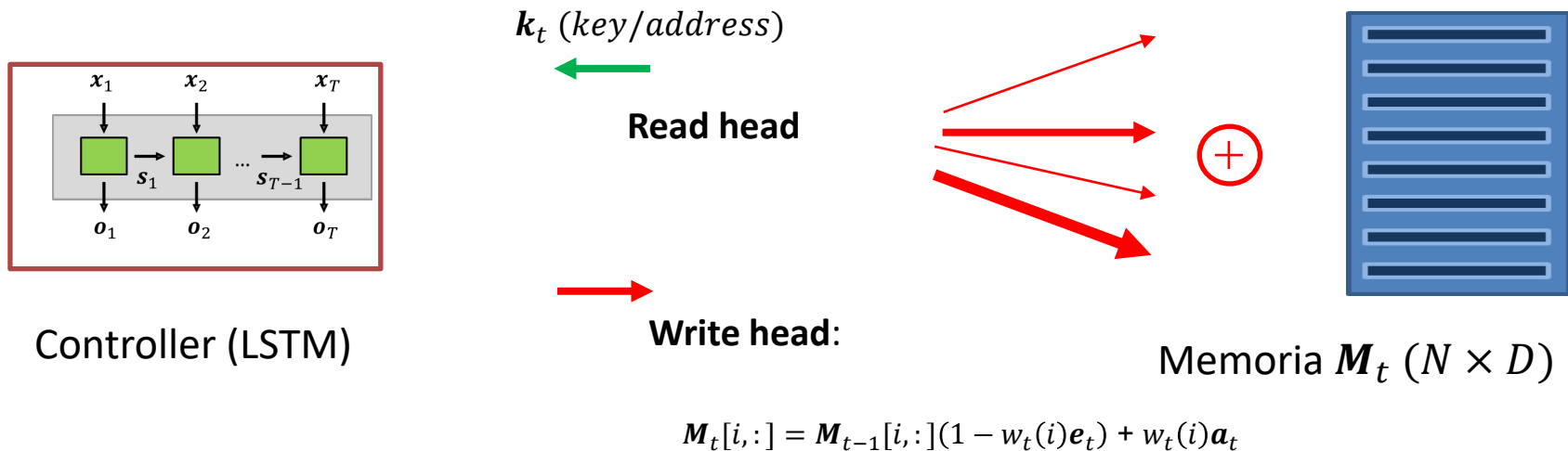
- Transformers: ¿Máquinas programables de propósito general?
 - “differentiable computer”



w_t peso/dirección de lectura/escritura se calcula por **parecido**

Transformers

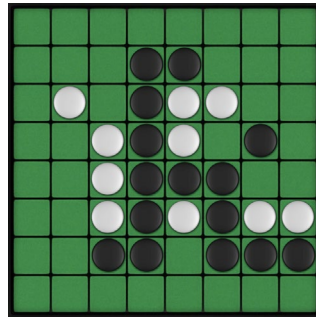
- Transformers: ¿Máquinas programables de propósito general?
 - “differentiable computer”



w_t peso/dirección de lectura/escritura se calcula por **parecido**, como veremos luego

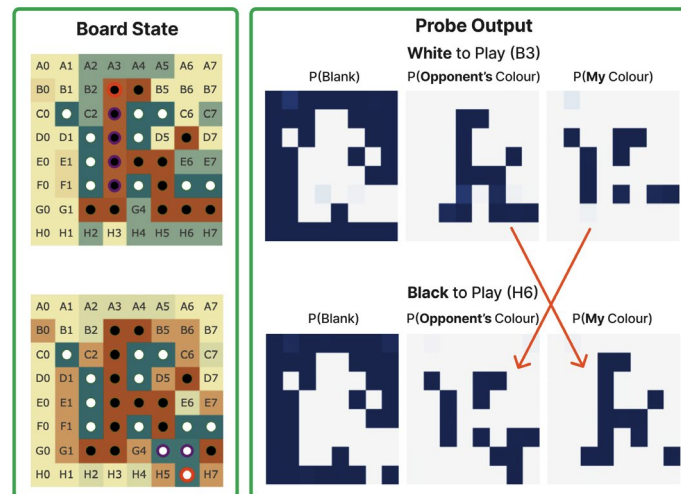
Transformers

- **Ejemplo** generación de modelos del mundo
 - El modelo se entrena para predecir en siguiente movimiento.
 - El modelo tiene una secuencia de movimientos
F4, F3, D2, F5, G2, F2, G3, C4, E5, F6, D6, E2, B4, C5, G7, C1, G6, F7, G5, C3, B3, **H6**
 - Si nos ayudamos de una representación visual entendemos mucho mejor de qué juego se trata y cuál es el estado actual del tablero



Transformers

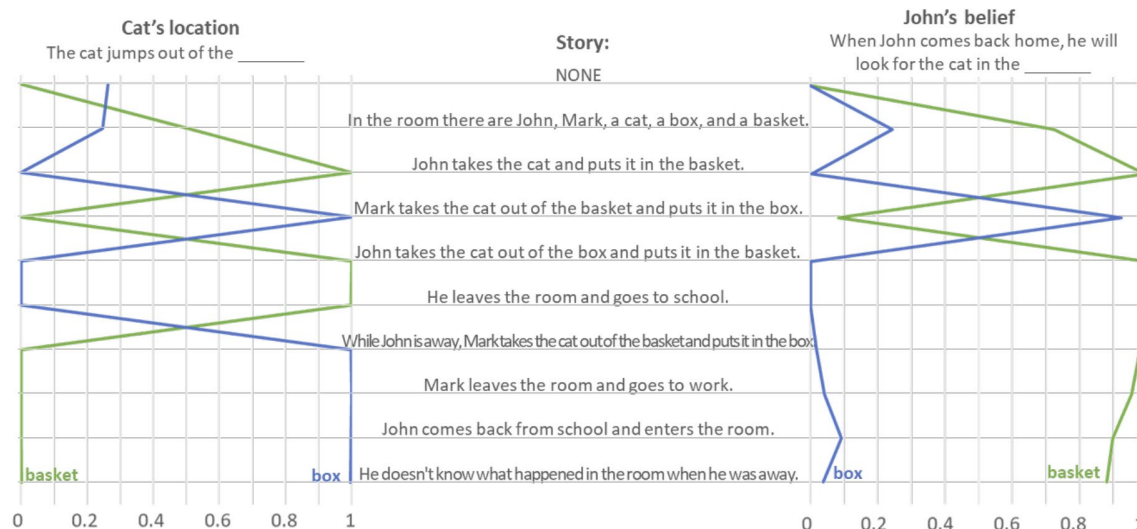
- **Ejemplo** generación de modelos del mundo
 - En ese ejemplo se muestra cómo procesando los vectores de representación.
 - El modelo representa internamente el mapa del tablero diferenciando el color de las piezas y a quién le toca jugar



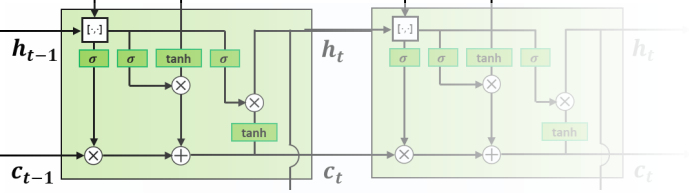
<https://www.lesswrong.com/posts/nmxzr2zsjNtjaHh7x/actually-othello-gpt-has-a-linear-emergent-world>

Transformers

- Transformers: Razonamiento / modelos del mundo
 - Existen numerosos trabajos recientes en los que se evalúan o entrenan LMs para resolver multitud de tareas



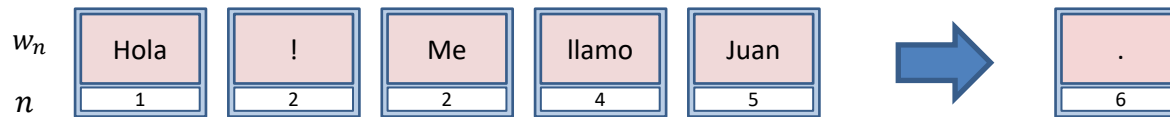
Theory of Mind May Have Spontaneously Emerged in Large Language Models:
Michal Kosinski



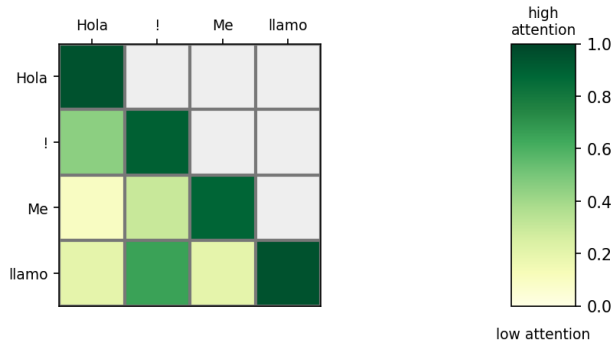
Modelos recientes

Modelos recientes

- Transformer



El número de cálculos crece con N^2



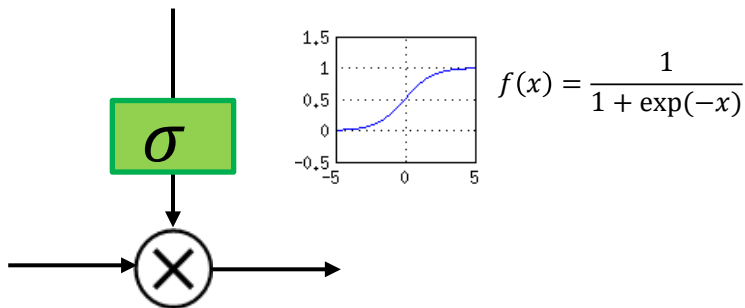
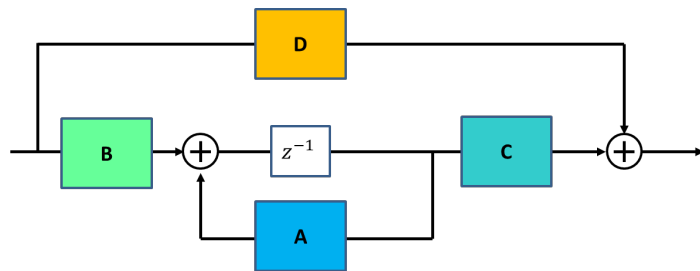
$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- Generar una secuencia larga se convierte en un problema
 - O dar una respuesta basada en un contexto muy amplio

Modelos recientes

- MAMBA: Linear-time sequence modeling with selective state spaces

Gu, A., & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.



- Alternativa a los Transformers para secuencias largas, con coste lineal en tiempo y memoria.
- Captura dependencias a largo plazo
- Facilitan el procesamiento *streaming* y en tiempo real gracias a su naturaleza recurrente y causal.
- Han abierto una nueva línea de investigación híbrida entre control, señales y deep learning aplicada a texto, audio y series temporales.

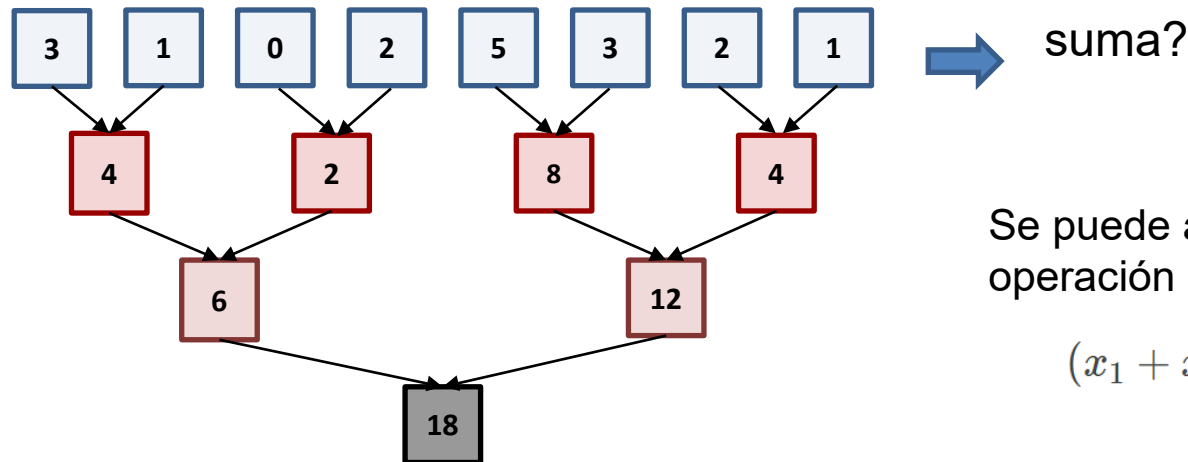
Modelos recientes

- Associative scan

<https://www.cs.cmu.edu/afs/cs/academic/class/15750-s11/www/handouts/PrefixSumBlelloch.pdf>

https://en.wikipedia.org/wiki/Prefix_sum

...



Se puede aplicar cuando la operación es asociativa:

$$(x_1 + x_2) + x_3 = x_1 + (x_2 + x_3)$$

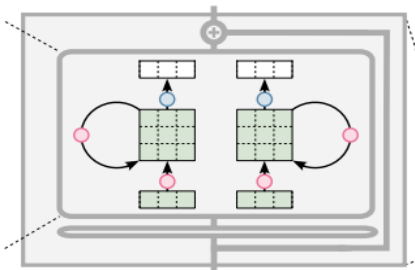
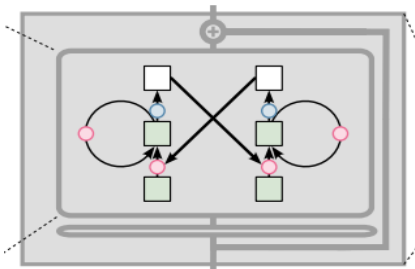
- Es un algoritmo para calcular sumas de una secuencia, **dividiéndola en partes** y combinando resultados.
- En el caso de los modelos de secuencia permite la **paralelización para GPU** (en entrenamiento).

Modelos recientes

- **xLSTM: Extended Long Short-Term Memory**

- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J. and Hochreiter, S., 2024. xLSTM: Extended Long Short-Term Memory. *arXiv preprint arXiv:2405.04517*. <https://github.com/NX-AI/xlstm.git>

xLSTM Blocks



- Uno de los autores originales de las LSTM ([Hochreiter](#)) ha actualizado los modelos recurrentes aprovechando los avances recientes -> **xLSTM**
- Muy buenos resultados en tareas de lenguaje, imágenes y **razonamiento** lógico.

Modelos recientes

- xLSTM

Ejemplos:

- **Even pairs:** Dado una secuencia de símbolos **a** y **b**, se comprueba si el número de **ab** y **ba** es par. Esto se puede resolver mirando si el primer y el último son iguales

Even Pairs

Sequence: a b b a a b a b a **a**

Transformer	LSTM	Mamba	xLSTM
100%	100%	100%	100%

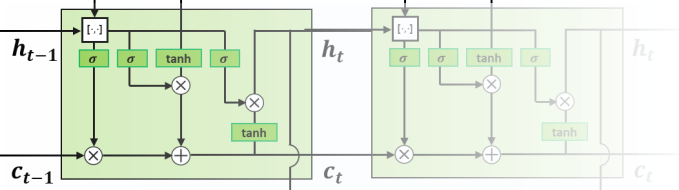
- **Parity:** Dado una secuencia binaria de tokens **a** y **b**, se comprueba si el número de **b** es par. Si es par se devuelve **a**, y si no, **b**

Parity:

Sequence: a b b a a b a **b**

Transformer	LSTM	Mamba	xLSTM
3%	100%	13%	100%

Modelos recientes



- **Mod arithmetic:** (Sin paréntesis) Calcula el resultado de las operaciones aritméticas del contexto usando módulo n.

Transformer	LSTM	Mamba	xLSTM
3%	100%	5%	100%

Mod Arithmetic (w/o Braces)
Sequence: 0 - 4 + 0 - 2 = 4 [PAD]

- **Majority Count:** Dada una secuencia, calcula el símbolo que más veces aparece en la secuencia.

Transformer	LSTM	Mamba	xLSTM
13%	45%	33%	46%

Majority Count
Sequence: 1 7 6 4 4 8 1 7 2 2

- **Solve equation:** Dada una ecuación con los operadores {+, -, ,, =, (,)}, números y una variable desconocida x, calcula el valor de x módulo el número máximo.

Transformer	LSTM	Mamba	xLSTM
2%	38%	3%	55%

Solve Equation:
Sequence: (((2 + 0) + - x) - (1)) = 2 [ACT] 2

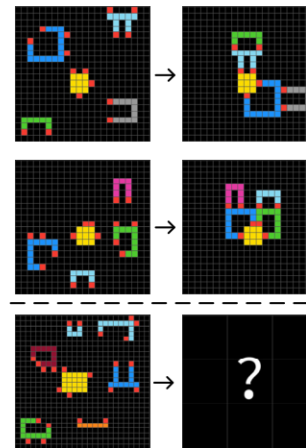
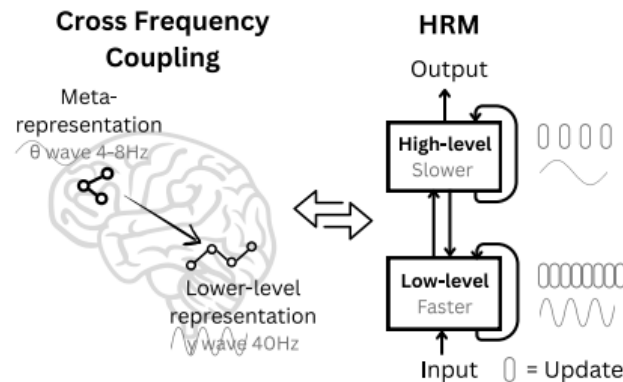
Modelos recientes

• HRM

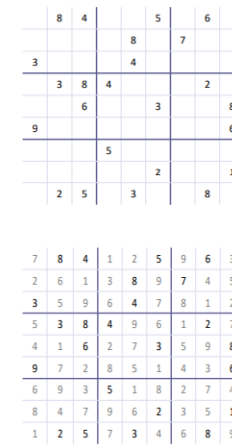
Hierarchical Reasoning Model

[Guan Wang](#), [Jin Li](#), [Yuhao Sun](#), [Xing Chen](#), [Changling Liu](#), [Yue Wu](#), [Meng Lu](#), [Sen Song](#), [Yasin Abbasi Yadkori](#) arXiv:2506.21734

- HRM es un modelo **recurrente jerárquico** inspirado en el procesamiento del cerebro a múltiples escalas de tiempo.
- Con dos módulos y pocos datos de entrenamiento, puede **resolver tareas complejas de razonamiento**.



(a) ARC-AGI

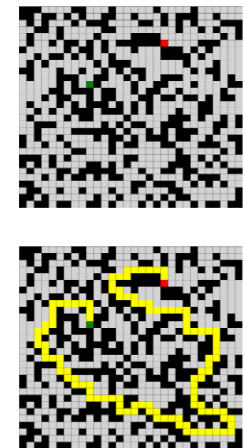


Two examples of Sudoku-Hard puzzles. The first is a 4x4 grid, and the second is a 9x9 grid.

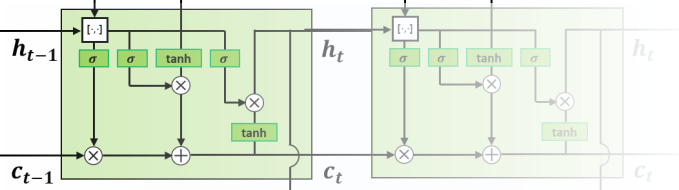
8	4		5	6	
3			8	7	
3	8	4			2
9	6		3		8
		5			6
2	5		2		1

7	8	4	1	2	5	9	6	3
2	6	1	3	8	9	7	4	5
3	5	9	6	4	7	8	1	2
5	3	8	4	9	6	1	2	7
4	1	6	2	7	3	5	9	8
9	7	2	8	5	1	4	3	6
6	9	3	5	1	8	2	7	4
8	4	7	9	6	2	3	5	1
1	2	5	7	3	4	6	8	9

(b) Sudoku-Hard



(c) Maze navigation



Modelos recientes

- Funciones recursivas
 - Una función recursiva se llama a sí misma con problemas más pequeños hasta llegar a un caso base.
 - Se usa en problemas de divide y vencerás y permite encontrar estados estables o soluciones de forma sencilla.

```
const fact = n => {
  if (n === 0) {
    return 1;
  }
  return n * fact(n - 1);
}
```

So `fact(4)` will result in the following computation:

```
fact(4)
4 * fact(3)
4 * (3 * fact(2))
4 * (3 * (2 * fact(1)))
4 * (3 * (2 * (1 * fact(0))))
4 * (3 * (2 * (1 * 1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
4 * 6
24
```

<https://medium.com/@claytoncripe/understanding-recursion-a-step-by-step-guide-3b13adf1f032>

<https://www.laurentluce.com/posts/solving-mazes-using-python-simple-recursivity-and-a-search/>

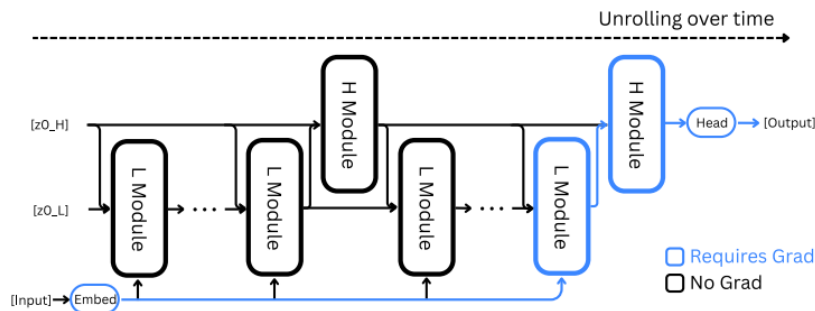
Modelos recientes

- HRM

Hierarchical Reasoning Model

[Guan Wang](#), [Jin Li](#), [Yuhao Sun](#), [Xing Chen](#), [Changling Liu](#), [Yue Wu](#), [Meng Lu](#), [Sen Song](#), [Yasin Abbasi Yadkori](#) arXiv:2506.21734

- Los HRM aplica dos **funciones recursivas**
 - una de bajo nivel para detalles y otra de alto nivel para planificación.
- El algoritmo repite la llamada a la función varias veces



```

def hrm(z, x, N=2, T=2):
    x = input_embedding(x)
    zH, zL = z
  
```

```

    with torch.no_grad():
        for _i in range(N * T - 1):
            zL = L_net(zL, zH, x)
            if (_i + 1) % T == 0:
                zH = H_net(zH, zL)
  
```

```

    # 1-step grad
    zL = L_net(zL, zH, x)
    zH = H_net(zH, zL)
    return (zH, zL), output_head(zH)
  
```

Modelos recientes

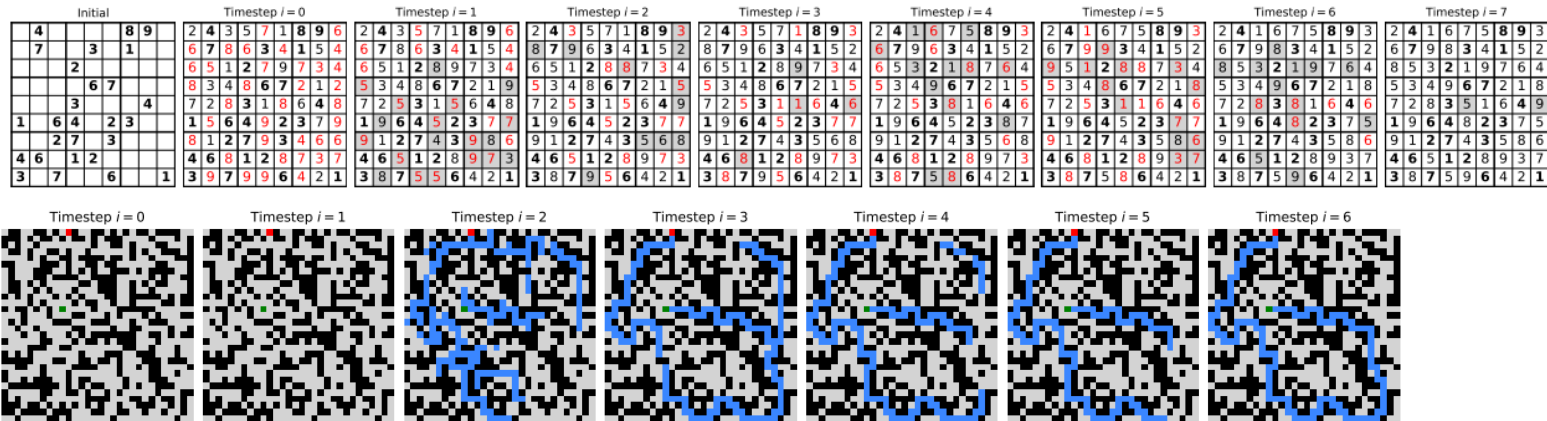
• HRM / TRM

Hierarchical Reasoning Model

[Guan Wang](#), [Jin Li](#), [Yuhao Sun](#), [Xing Chen](#), [Changling Liu](#), [Yue Wu](#), [Meng Lu](#), [Sen Song](#), [Yasin Abbasi Yadkori](#) arXiv:2506.21734

Less is More: Recursive Reasoning with Tiny Networks

[Alexia Jolicoeur-Martineau](#)



– Resultados en solución de sudokus y laberintos:

	sudokus	laberintos
HRM (27)	55.0%	74.5%
TRM (7M)	74.7%	85.3%
deepseek-r1	0.0%	0.0%
o3-mini-high	0.0%	0.0%

Modelos recientes

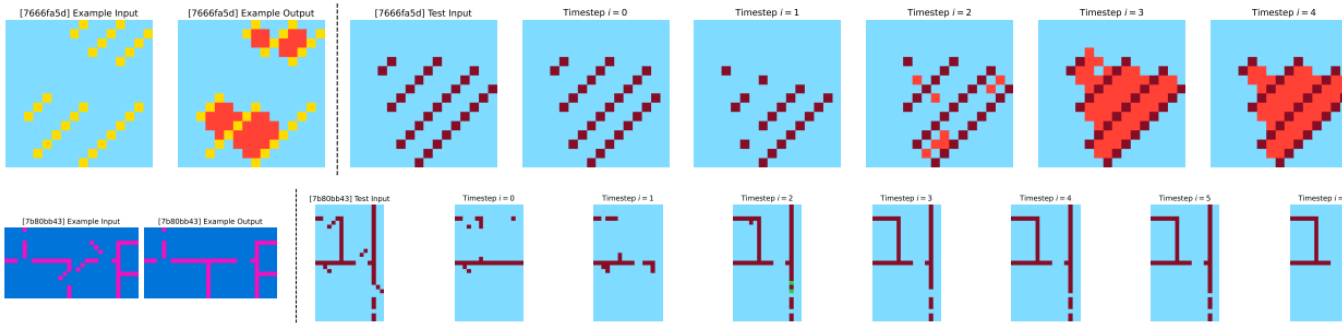
• HRM / TRM

Hierarchical Reasoning Model

[Guan Wang](#), [Jin Li](#), [Yuhao Sun](#), [Xing Chen](#), [Changling Liu](#), [Yue Wu](#), [Meng Lu](#), [Sen Song](#), [Yasin Abbasi Yadkori](#) arXiv:2506.21734

Less is More: Recursive Reasoning with Tiny Networks

[Alexia Jolicoeur-Martineau](#)



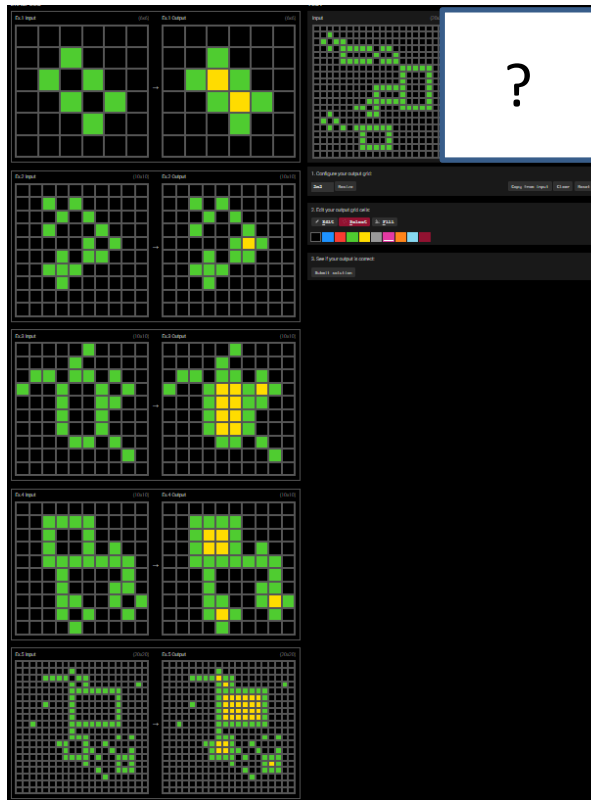
– En tareas de lógica ARC-challenge 1 y 2

	ARC1	ARC2
Humanos	98.0%	100.0%
TRM (7M)	44.6%	7.8%
deepseek-r1	15.8%	1.3%
o3-mini-high	34.5%	3.0%
Gemini 3	87.5%	54.0%
chatgpt 5.2	85.7%	54.2%

Modelos recientes

- ARC challenges: ARC-AGI1

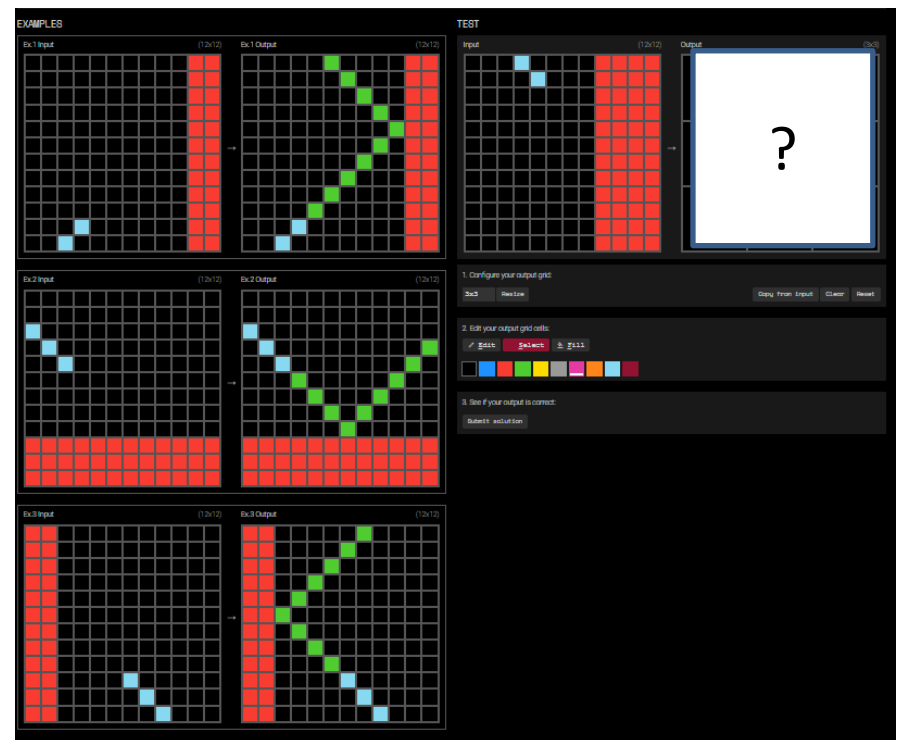
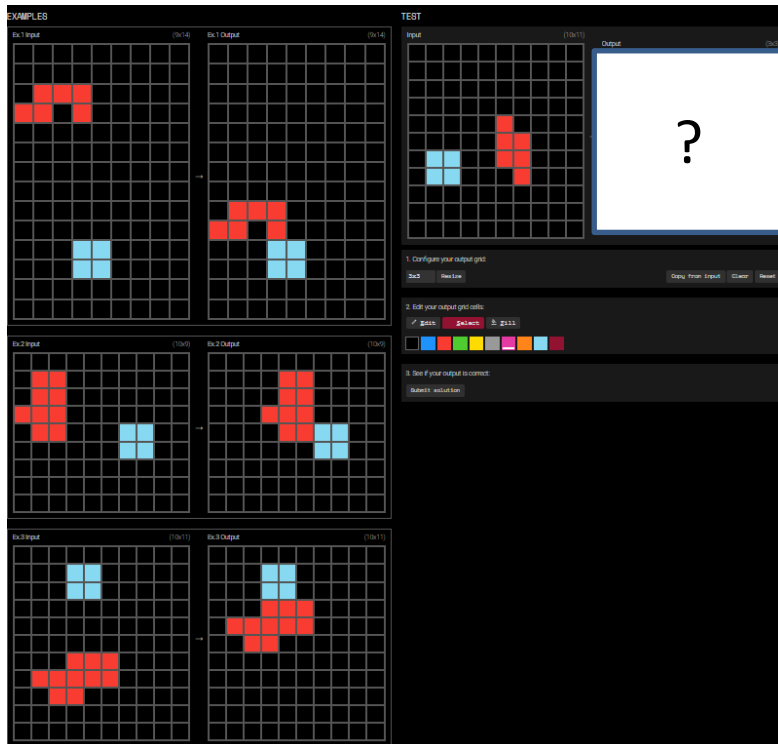
<https://arcprize.org/arc-agi/1/>



Modelos recientes

- ARC challenges: ARC-AGI1

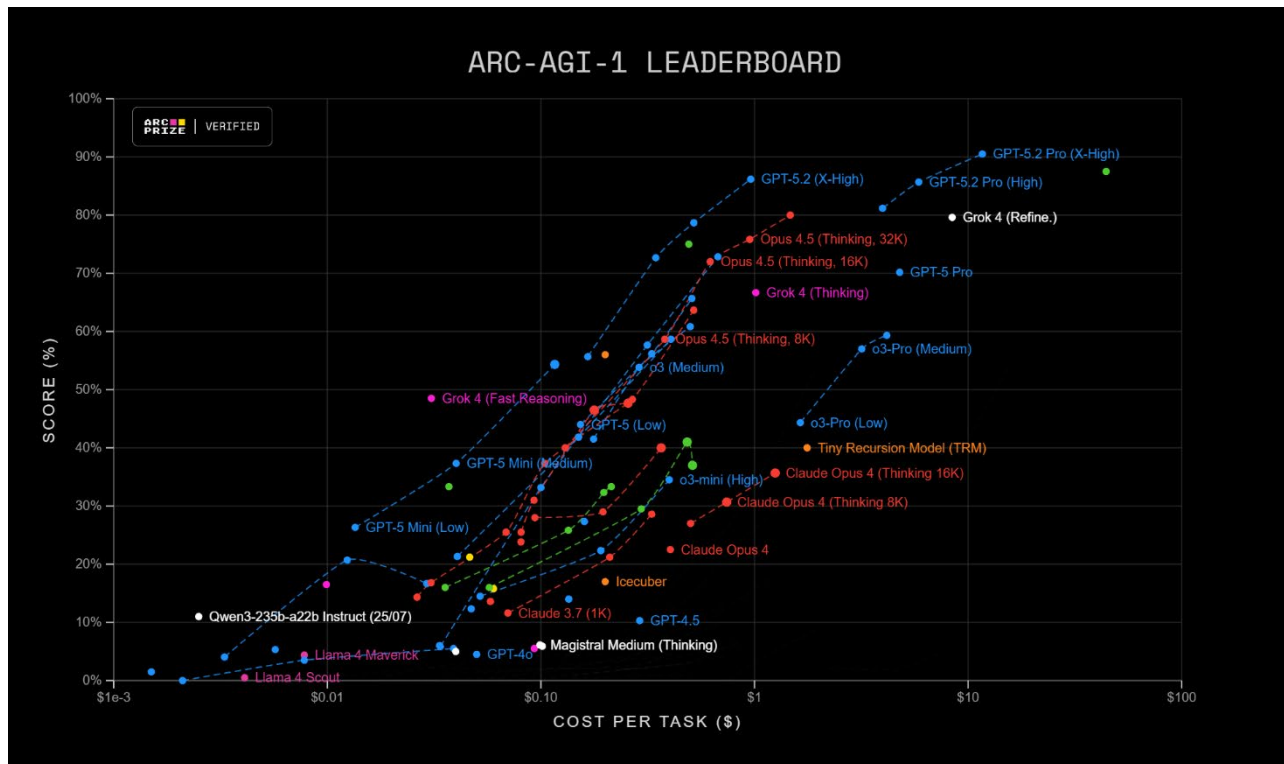
<https://arcprize.org/arc-agi/1/>



Modelos recientes

- ARC challenges: ARC-AGI1

<https://arcprize.org/arc-agi/1/>



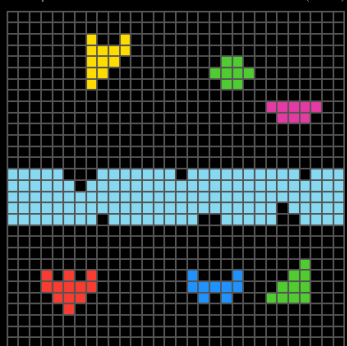
Modelos recientes

- ARC challenges: ARC-AGI2

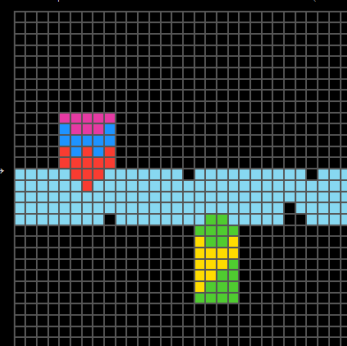
<https://arcprize.org/arc-agi/2/>

EXAMPLES

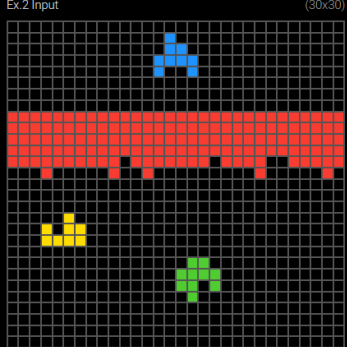
Ex.1 Input (30x30)



Ex.1 Output (30x30)



Ex.2 Input (30x30)

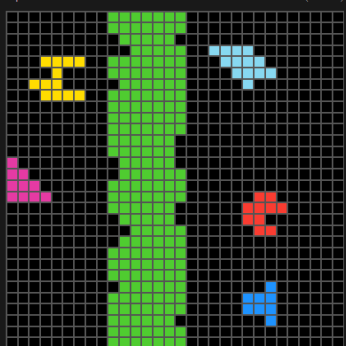


Ex.2 Output (30x30)

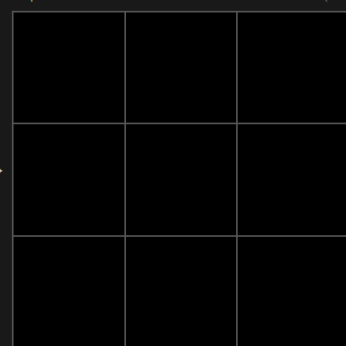


TEST

Input (30x30)



Output (3x3)



1. Configure your output grid:

3x3

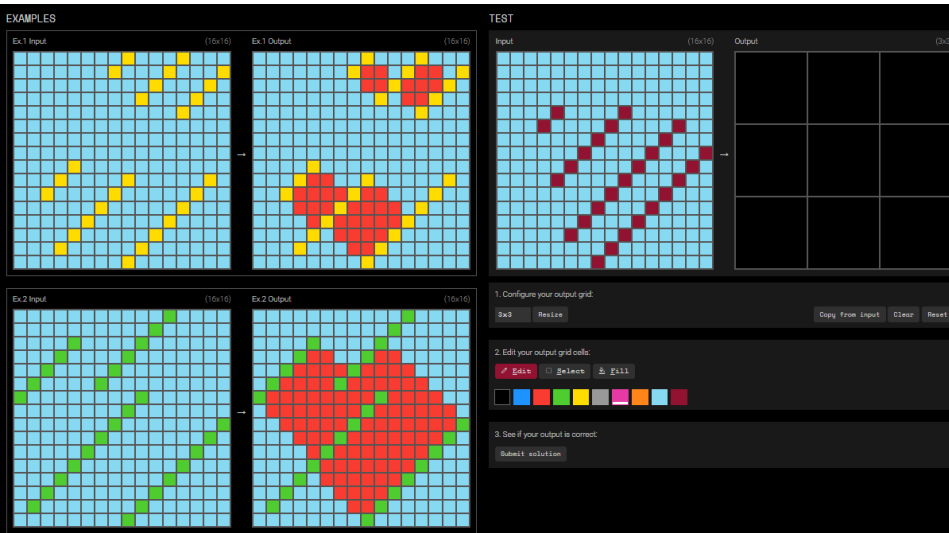
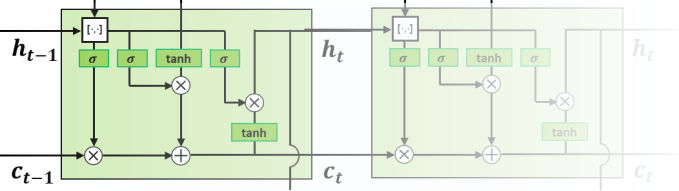
2. Edit your output grid cells:

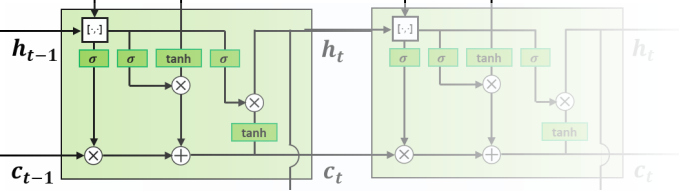
3. See if your output is correct:

Modelos recientes

- ARC challenges: ARC-AGI2

<https://arcprize.org/arc-agi/2/>

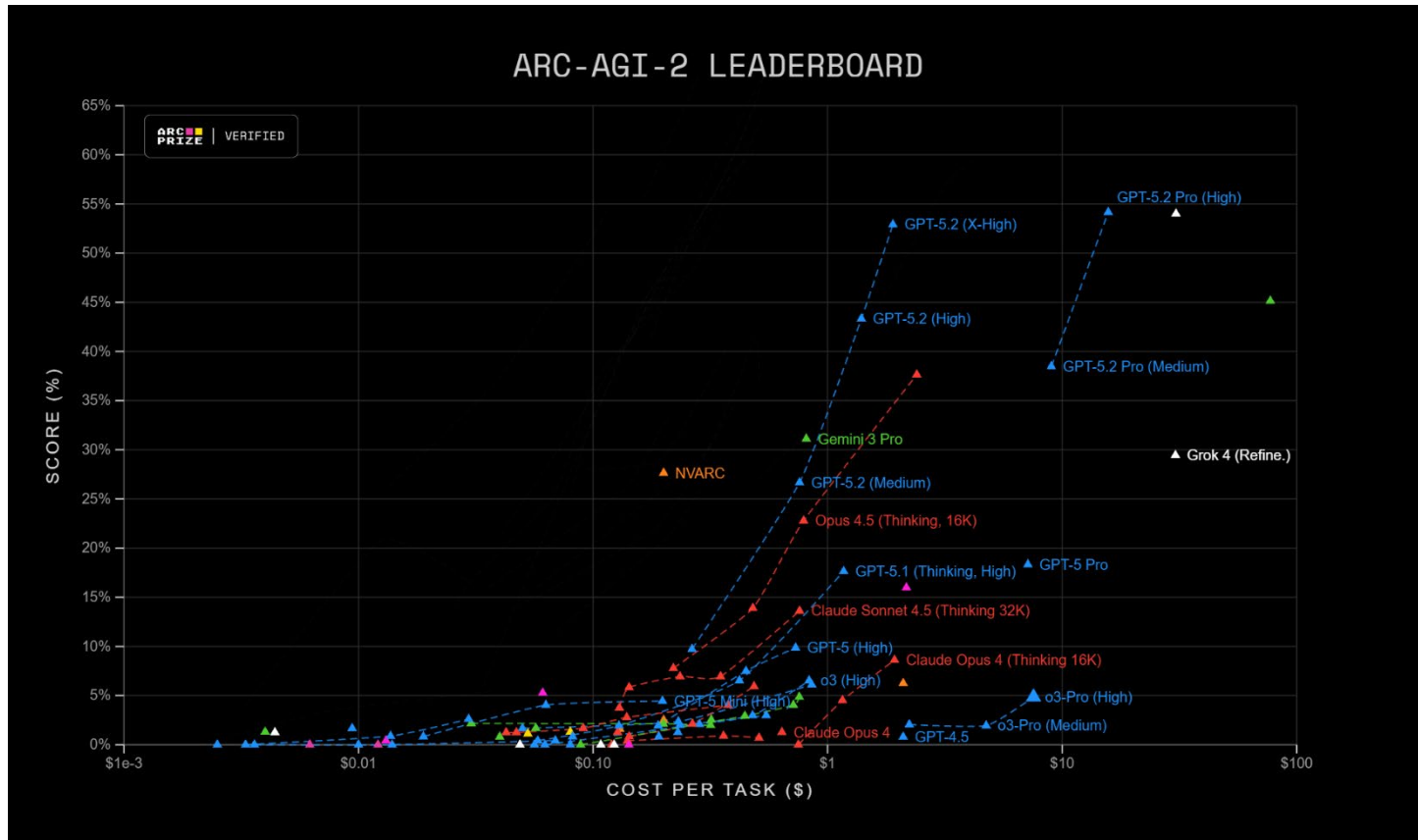




Modelos recientes

- ARC challenges: ARC-AGI2

<https://arcprize.org/arc-agi/2/>



Modelos recientes

- ARC challenges: ARC-AGI3

<https://arcprize.org/arc-agi/3/>

